

Frammentazione dei dati: usare l'AI per affittare il debito tecnologico o estinguerlo

Scritto con l'assistenza di un modello linguistico e tracciato con il metodo Colophon. La nota sul metodo, con le percentuali di contributo, è in fondo.

Ecco due situazioni tipiche.

Un'azienda di servizi B2C utilizza diversi software per automatizzare il ciclo attivo — lead generation, contatto, vendita, fatturazione, pagamento, erogazione del servizio — con una integrazione minimale tra di essi. Man mano che il business cresce i sistemi vengono fatti evolvere in maniera rapida, creando dei silos applicativi, ognuno con i propri dati, e la correlazione tra i dati dei vari silos non è più univoca o garantita, rendendo difficoltosa la governance end-to-end di processo (il pagamento ricevuto via bonifico a quale cliente e a quale lead si riferisce?) e le attività strategiche di evoluzione del business (quella campagna quanto ha generato in termini di incassato?). In questo caso la frammentazione dei dati introduce una inefficienza e può penalizzare la scalabilità dell'azienda.

Un'azienda di servizi B2B consolidata, che eroga importanti volumi di transazioni mission critical, al verificarsi di problemi operativi, di privacy e di sicurezza, non riesce a valutare in maniera tempestiva l'impatto degli stessi sulle persone (quanti clienti, quanto personale interno e di partner, e in quale ruolo), sui dati (quali dati, in che forma, quali trattamenti) e sui processi (quali azioni intraprendere, in che tempi, con che livello di autorizzazione). Questo perché il numero dei sistemi coinvolti, la loro stratificazione e sovrapposizione ha creato silos di dati, barriere, duplicazioni funzionali, spesso non adeguatamente documentate, soprattutto nel caso di dati non recenti. Estrarre un elenco di clienti impattati, una mappa degli attori aziendali coinvolti, fare un'analisi analitica dei dati e valutare l'impatto di un problema diventa di per sé un progetto che può durare settimane o mesi, con risultati e costi incerti. In questo secondo caso la frammentazione dei dati produce una storia non più dimostrabile: il rischio si accumula con i volumi di transazioni effettuate nel tempo. Con l'attuale forcing sulla compliance e sul controllo dei rischi, avere questo tipo di situazioni è una vera e propria bomba a orologeria pronta a esplodere.

Sono due facce dello stesso problema, e vale la pena nominarlo. Il "debito tecnologico" — tutte quelle situazioni per cui la piattaforma tecnologica di un'azienda è indietro rispetto allo stato dell'arte — è uno dei temi quotidiani di un CTO e di un CIO, e ha nella frammentazione e nella qualità del dato una delle sue componenti più costose. Quello che cambia, tra i due casi, è la valuta in cui lo si paga: nel primo efficienza, nel secondo rischio.

Il canone invisibile: cosa costa già oggi, e perché ora si vede

La frammentazione dei dati è un costo, spesso invisibile. Aziende in situazioni simili a quelle sopra descritte pagano sicuramente un costo importante per gestire le inefficienze che ne derivano: personale e

consulenti dedicati ad attività di riconciliazione manuale, o alla scrittura di procedure software di "integrazione" tra sistemi.

È una voce che non compare in nessun cruscotto, perché non è concentrata da nessuna parte: è un po' di tempo di molte persone, in molte funzioni diverse. La mia impressione è che in una struttura ICT corporate valga una quota degli opex a due cifre. Ma è esattamente il punto: nessuno lo sa con precisione, perché nessuno lo misura.

Quando questi costi emergono e diventano visibili? Un caso tipico, in passato, era la migrazione a un nuovo sistema. Attivare un nuovo CRM o un ERP è emblematico, perché costringe a fare ordine nei dati: spesso la parte più costosa e più lunga del progetto non è personalizzare il software nuovo, ma la bonifica dei dati da importare da quello vecchio. Una grande, immensa bonifica.

Un caso "nuovo" è dato dall'uso dell'AI non per risolvere alla fonte la frammentazione dei dati, ma per renderla tollerabile. Al posto di riconciliare a mano, al posto di migrare verso sistemi che coprono end-to-end tutti i processi, si costruiscono una serie di procedure "intelligenti" che riconciliano i dati. In apparenza sembra l'uovo di Colombo: l'AI è in grado di gestire anche situazioni nuove e non pienamente definite, sembra fatta apposta per interpretare i disallineamenti tra sistemi.

Ed è vero, è una soluzione efficace. Ma con due problemi. Primo, il problema non è risolto ma affittato: genera un costo ricorrente "in token" che può diventare importante — e che, a differenza del precedente, si vede. Secondo, è una soluzione non deterministica, nel senso che può sbagliare, e in alcuni contesti l'approssimazione aumenta il rischio invece di ridurlo.

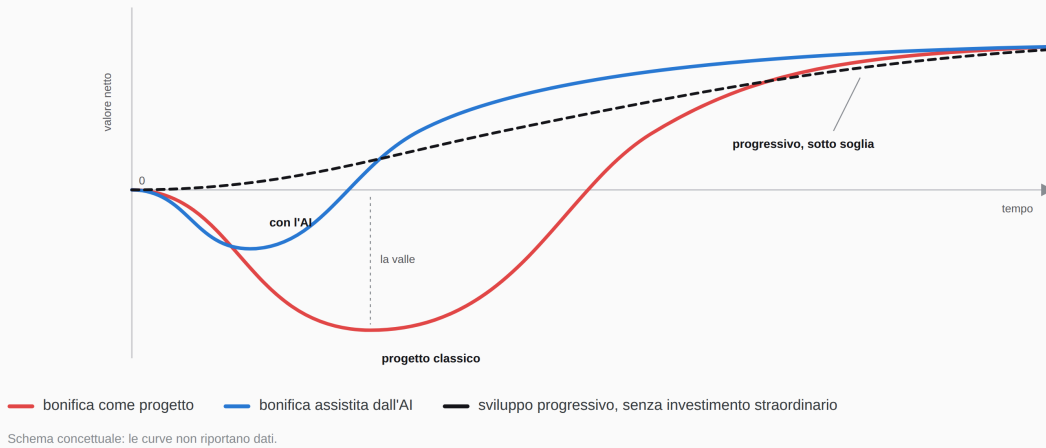
Perché nessuno lo risolve: la valle della disperazione, e la conversazione che si vince solo dopo il danno

Penso che qualsiasi CTO sappia trovare, per la propria azienda, un percorso per arrivare ad avere un'architettura del dato integrata e consistente, con una implementazione performante, resiliente, sicura e scalabile.

Ma come si fa a sostenere questo tipo di progetto di fronte a un CEO, a uno steering committee o a un CDA? I benefici in termini di riduzione del costo e del rischio possono essere importanti, ma in generale si parte da una situazione iniziale in cui il costo operativo è nascosto e l'azienda comunque funziona. Il tema è poi squisitamente tecnico, e impegnare l'azienda per avere un risultato nel medio periodo, passando comunque attraverso la "valle della disperazione" tipica di ogni progetto, è una scelta che rischia di isolare il CTO che la propone.

Il problema non è mai stato il ritorno. È la forma della curva.

Ogni progetto di bonifica passa da un periodo in cui l'azienda sta peggio di prima. È quel tratto — non il business case — a rendere la conversazione con il CEO impossibile da vincere.



La valle della disperazione. L'ostacolo non è il ritorno dell'investimento, è il tratto in cui l'azienda sta peggio di prima.

L'unica situazione in cui tutti gli stakeholder alzano il proprio livello di commitment, paradossalmente, è a fronte di un problema. Se il rischio accumulato a un certo punto "esplode" in un incidente, il piano di remediation può diventare il modo per risolvere il debito tecnologico che lo ha causato. Il che significa che il momento in cui il progetto diventa approvabile è esattamente quello in cui ha smesso di essere un investimento ed è diventato una riparazione.

Cosa si rompe davvero: il debito semantico, la suite come soluzione impacchettata

Vale la pena essere precisi su cosa si rompe, perché non è quello che sembra. La difficoltà non è tecnica: formati, protocolli e connettori li risolviamo da vent'anni. La difficoltà è semantica. "Cliente", nel CRM, è un'opportunità che si è chiusa; in fatturazione è un soggetto giuridico con una partita IVA; in assistenza è chi apre il ticket. Sono tre entità con tre cicli di vita diversi, che a volte coincidono e a volte no. Il lead e il cliente nascono con chiavi diverse in sistemi diversi, e il legame tra quelle chiavi è spesso l'unica cosa che nessuno ha progettato, perché al momento sembrava ovvia. Quando quel legame si perde — ed è esattamente ciò che succede nel primo dei due casi — non hai perso un dato: hai perso la possibilità di ricostruire una storia. I progetti di integrazione non falliscono sull'ETL. Falliscono in riunione, quando bisogna decidere quale delle tre definizioni di "cliente" vince.

Il problema della frammentazione dei dati si risolve strutturalmente ricostruendo le correlazioni tra silos applicativi. Se ci pensate, è quello che succede quando un'azienda acquista un CRM o un ERP: alla fine non stai comprando un software, stai comprando la sua architettura del dato. È una soluzione impacchettata — per averla devi prendere anche la piattaforma, la migrazione e il lock-in. Ed è una scelta che si giustifica solo se ci sono driver più forti: efficienza operativa, governance dei processi, copertura

funzionale. La soluzione della frammentazione arriva come effetto collaterale, non è mai il motivo per cui si firma.

Tuttavia oggi l'AI spacchetta il pacchetto: è possibile ricostruire l'architettura del dato senza adottare piattaforme trasversali.

Cosa cambia adesso: l'AI accorcia il viaggio nella valle della disperazione

Prima di entrare nel merito, una distinzione che tiene insieme tutto il ragionamento: **l'AI nel percorso di costruzione, non nel percorso di esecuzione.**

Usare un modello per riconciliare i dati a ogni transazione significa metterlo nel percorso di esecuzione: il costo è un canone che cresce con i volumi, l'errore accade in produzione su un dato vero, e l'output è la risposta stessa. Usarlo per progettare l'architettura, bonificare lo storico e scrivere le procedure di accesso significa metterlo nel percorso di costruzione: il costo è una tantum, l'errore si scopre in revisione, e l'output è deterministico — codice, schemi, migrazioni.

È la stessa tecnologia. Cambia se la compri come investimento o la affitti come servizio.



Lo stesso modello, due collocazioni diverse: dentro ogni transazione, oppure a monte a produrre artefatti deterministici.

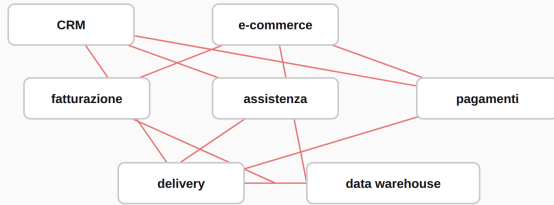
Al posto di usare l'AI per costruire procedure di integrazione "intelligenti" tra sistemi, è possibile implementare una strategia diversa.

1) Disegnare un'architettura del dato, cioè un modello semantico di riferimento e uno strato di accesso comune. Non è lo schema di un database: è l'insieme delle regole su quale sia la fonte certa di ogni informazione, come ci si accede, e chi decide quando cambia.

Non è lo schema di un database: è chi decide qual è la fonte certa.

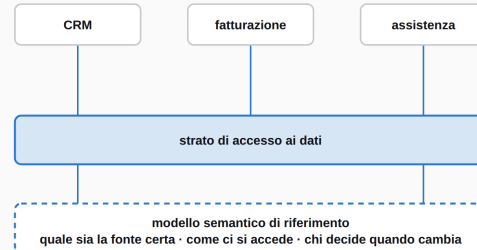
A sinistra ogni sistema parla con ogni altro, e la correlazione fra i dati non è più garantita. A destra tutti parlano con uno strato di accesso comune, governato da un modello semantico di riferimento.

Oggi integrazioni punto a punto



nessuna fonte certa · duplicazioni · legami di chiave che si perdono

Con l'architettura del dato



governato da un team di agenti · ogni nuovo componente passa di qui

Da integrazioni punto a punto a uno strato di accesso comune, governato da un modello semantico di riferimento.

Può essere, nei casi più semplici, un unico database aziendale, opportunamente ridonato e con tutte le misure di sicurezza del caso. Nel caso più generale è un insieme di componenti (diversi database, storage, servizi di accesso, middleware di comunicazione) che garantiscono correttezza, unicità, distribuzione e sicurezza del dato, e lo rendono accessibile dove serve con prestazioni adeguate ai singoli casi d'uso. La cosa importante è avere una governance unica e un processo chiaro, robusto e veloce per farlo evolvere alla velocità del business. Le stesse applicazioni aziendali sono parte di questa architettura: se la fonte certa di un dato — per esempio l'anagrafica clienti — è il CRM aziendale, l'architettura del dato garantirà che ogni copia di quel dato in altri sistemi sia slave della fonte di verità aziendale.

2) Nel breve periodo, tutte le procedure di integrazione tra sistemi (che sicuramente esistono) vengono disintermediate da procedure di dialogo tra ogni sistema e lo strato di accesso ai dati comune.

3) Nel medio periodo, i singoli sistemi usano lo strato di accesso ai dati come riferimento per la gestione di qualsiasi informazione il cui ambito di utilizzo sia più esteso del proprio dominio applicativo.

Ora, questo approccio — oserei dire "classico" — ha un impatto pesante sull'architettura ICT di un'azienda. Ma essendo la difficoltà di tutte queste integrazioni di natura semantica, è proprio il campo dove gli LLM possono dare un contributo determinante. In particolare:

Il modello semantico di riferimento può essere molto articolato: alcune scelte dipendono da analisi approfondite dei dati (per esempio eliminare le ridondanze in un'anagrafica clienti) e le procedure di bonifica possono essere molto complesse. La progettazione e la manutenzione del modello possono essere affidate a un set di agenti AI specializzati, che mantengono ed evolvono non solo il modello ma soprattutto il contesto che ci sta dietro, ovvero il vero know how aziendale sui dati operativi.

Realizzare le procedure di integrazione con i vari sistemi e fornire uno strato di accesso ai dati è un'attività che richiede lo sviluppo di una mole rilevante di codice. Anche in questo caso l'analisi e lo sviluppo

possono essere affidati ad agenti AI, con costi e tempi che non sono comparabili a quelli di un team tradizionale.

Gli agenti demandati alla gestione dell'architettura possono poi trovare, realizzare e gestire tutte le ottimizzazioni possibili: ridurre il numero di servizi esposti, migliorare prestazioni, costi di esercizio e sicurezza.

L'evoluzione del modello semantico e dello strato di accesso può essere gestita in maniera analoga. Immaginate un team che vuole realizzare una nuova applicazione, che ha necessità di dati aziendali già esistenti e ne aggiunge di nuovi: l'agente architect riceve le specifiche, genera una proposta di soluzione — dati più servizi di accesso — riutilizzando al massimo quanto già disponibile, o attivando un agente developer per realizzare eventuali nuovi componenti.

Il rovescio: la frattura di velocità, il vibe coding

Pensando ad aziende che decidono di spostare il baricentro sullo sviluppo make di tutta o parte della propria piattaforma applicativa, la frammentazione dei dati è un grandissimo rischio.

Di solito l'attenzione è focalizzata sulle funzionalità applicative, e i dati vengono clonati o trasportati. In sintesi: la strategia del dato non evolve alla stessa velocità con cui si producono nuove funzionalità, artifact e componenti. È una vera e propria frattura di velocità, e non esistono colpevoli: nessuno decide consapevolmente di frammentare, è che un lato del sistema ha accelerato di dieci volte e l'altro no.

Sulla frammentazione dei dati non esistono numeri pubblici; sul codice sì, ed è la stessa frattura vista dal lato in cui qualcuno tiene il conto. GitClear misura la qualità del software leggendo la storia dei repository git: nella ricerca del 2026 ha analizzato 623 milioni di modifiche fatte fra il 2023 e il 2026 — il periodo in cui gli assistenti AI sono entrati nello sviluppo — confrontandole con gli anni precedenti. Due indicatori dicono che si produce molto e si riordina poco: la duplicazione di blocchi di codice è cresciuta dell'81%, e il refactoring — le righe spostate anziché aggiunte — è sceso dal 21% del 2022 al 3,8%. Ma i due che descrivono meglio quello di cui sto parlando sono altri: la quota di modifiche che toccano codice non aggiornato da oltre un anno è scesa del 74%, e il numero di chiamate con cui il codice nuovo si aggancia a quello esistente è calato del 35%. Tradotto: nessuno torna più indietro a sistemare, e il nuovo nasce già scollegato. È il silo, misurato.

(È l'analisi di un fornitore, non uno studio peer-reviewed, e non dichiara i propri limiti metodologici. La prendo come indizio robusto, non come prova — ma è un indizio che chiunque stia sviluppando in questo modo riconoscerà.)

Il vibe coding non ha inventato il problema della frammentazione: ha tolto l'attrito che lo frenava.

Realizzare un'architettura del dato unica e governata da agenti AI risolve nativamente questo problema. Nello sviluppo in vibe coding si attiva l'architect virtuale in charge dell'architettura del dato, che produce quanto necessario all'applicazione. E in questo caso l'AI non bonifica i dati: lavora direttamente sulle procedure di accesso, creazione e modifica. La scelta su come gestire i dati non è dell'applicazione, ma del

Questo approccio si può applicare anche ad altri temi architetturali: la frammentazione dei dati è il caso che tratto qui, ma un tema analogo riguarda per esempio la gestione delle identità e le procedure di autenticazione e autorizzazione.

Un punto importante riguarda la gestione del rischio. Se pensate ai due esempi iniziali, alla fine la frammentazione dei dati genera costi che nel "business as usual" sono assorbiti e accettati dall'azienda, e spesso giustificati dalla velocità di implementazione o dalla complessità di gestione. Quindi, pur esistendo aree di miglioramento, dal punto di vista dei costi può anche essere che non ci sia alcun allarme: tutti i cruscotti vanno bene.

IL RISCHIO CHE NON SI VEDE

Il costo lo misuri. Il rischio cresce lo stesso, e si abbatte solo dopo il danno.

L'incidente non fa aumentare il rischio: è il momento in cui il rischio già accumulato si manifesta. Il piano di remediation lo riduce davvero — ma non tocca il meccanismo che lo genera, e da lì ricomincia a salire.

Il diagramma è diviso in due pannelli che condividono un asse del tempo orizzontale.

Pannello superiore (Costo):

- Costo:** Una linea blu continua che cresce linearmente nel tempo.
- Descrizione:** "cresce piano - è misurato - i cruscotti sono verdi".
- Nota:** "nessun allarme, prima né dopo".

Pannello inferiore (Rischio):

- Rischio accumulato:** Una linea rossa tratteggiata che cresce esponenzialmente nel tempo.
- Descrizione:** "cresce con i volumi - non è misurato - nessuno lo vede".
- Incidente:** Un punto rosso sulla curva del rischio, indicato da una freccia. Sotto di esso c'è il testo: "l'incidente non aumenta il rischio: lo rende visibile".
- Piano di remediation:** Una freccia indica la parte della curva del rischio che scende dopo l'incidente, con il testo: "piano di remediation il debito che lo ha causato viene ridotto".
- Nota:** "la curva esiste anche se nessuno la disegna".

Schema concettuale: le curve non riportano dati. I due pannelli condividono l'asse del tempo, non la scala. Il tratteggio indica una grandezza che l'azienda non sta osservando; il tratto pieno il periodo in cui, dopo l'incidente, la guarda.

Penso sia importante, nell'analisi dei propri dati, rilevare questo tipo di informazioni:

- 7

- quale quota dei flussi critici attraversa più di N sistemi;
- il costo di riconciliazione per transazione — che ora, in token, si può davvero calcolare.

Sono quattro grandezze molto diverse, e non ho una formula da proporre: chiunque ve ne proponga una, a questo stadio, ve la sta vendendo. Quello che si può fare subito è più modesto e più utile. Prendete i tre o quattro flussi che generano più valore e, per ciascuno, contate quanti sistemi attraversa, quanti legami di chiave non sono ricostruibili in automatico, e quanto costa oggi riconciliarli. Sono numeri che si ottengono in qualche giorno e che raramente esistono già da qualche parte. Il primo valore di quell'esercizio non è l'indice: è che per la prima volta qualcuno in azienda ha un numero da mettere accanto a una parola che finora era solo un'impressione.

Da dove si comincia: l'architettura to-be come bersaglio e come cancello

La prima cosa da fare è definire — con il supporto dell'AI e usando tutte le informazioni a disposizione dell'azienda — l'architettura del dato to-be: il modello semantico di riferimento e lo strato di accesso.

La realizzazione è poi incrementale, ed è l'unico approccio sensato quando si sviluppa in *vibe coding*. Procede in due direzioni opposte: una guarda indietro, l'altra guarda avanti.

1) Il bersaglio: bonificare l'esistente. Si identificano tutti i punti dove la correlazione dei dati è problematica e lì si indirizza uno per uno, usando l'architettura del dato come strumento di disintermediazione. L'architettura dice dove si deve arrivare, e ogni intervento accorcia la distanza.

2) Il cancello: impedire che il nuovo aggiunga debito. Al posto di realizzare componenti che duplicano dati e creano nuovi flussi — e che quindi, paradossalmente, aumentano la frammentazione — gli agenti che governano l'architettura del dato vengono coinvolti nello sviluppo di ogni nuovo oggetto. Nessun componente nuovo entra in produzione senza essere passato di lì: è questo che ferma l'emorragia, e comincia a farlo dal primo giorno.

In questo modo si cominciano ad avere risultati anche nel breve termine. Con un uso attento di un'architettura ad agenti gli sviluppi diventano progressivi, e il costo può rimanere "sotto soglia", senza dover chiedere un investimento straordinario. È questo che rende l'approccio difendibile davanti al CEO: produce effetto prima che la bonifica sia finita. Il che, guarda caso, non accorcia più il viaggio nella valle: lo evita del tutto.

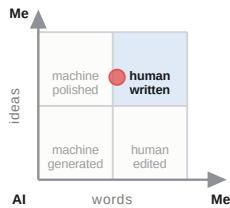
Pensando alle due situazioni aziendali citate all'inizio:

L'azienda cresciuta in fretta costruirebbe la propria architettura del dato con gli stessi strumenti con cui costruisce tutto il resto — in *vibe coding* — ma con obiettivi diversi dalla sola velocità: governance piena, correttezza, tempestività di accesso al dato. Non un progetto separato: una regola su come si sviluppa.

L'azienda consolidata può partire dalla sola riduzione dei costi operativi, senza generare traumi organizzativi e con risultati costanti e misurabili a ogni passo. E proprio per questo costruire il consenso

dal basso, invece di doverlo chiedere dall'alto. Che è, di nuovo, un modo di attraversare la valle senza scenderci dentro.

In estrema sintesi: la frammentazione dei dati non è mai stata un problema di fattibilità tecnica, ma di sostenibilità del progetto. Usare l'AI non serve a rendere il debito più tollerabile: serve a rendere la sua estinzione finalmente proponibile. Chi la userà solo per tollerarlo, lo pagherà a canone, per sempre.



Nota sul metodo. Ho scritto questo articolo con l'assistenza di un modello linguistico, e ogni intervento è stato registrato mentre accadeva con il metodo Colophon. Contenuto: 69% mio, 31% dell'AI. Testo: 53% mio, 47% dell'AI. I due numeri misurano cose diverse — il primo le idee, il secondo le parole che le esprimono — e la differenza è la parte interessante: l'AI ha scritto più parole di quante idee abbia portato, perché è intervenuta soprattutto in revisione, ricerca e titolazione. La prima stesura è mia all'86%. Il quadrante qui accanto colloca il testo sui due assi. Il registro è pubblicato, firmato e ispezionabile: github.com/fchinaglia/colophon, in cases/002. Di ogni affermazione rispondo io.

Registro: 81 eventi, radice ae68ae8d...88312793. Firma Ed25519 e marca temporale accanto al registro; istruzioni di verifica in VERIFY.md.